

Deployment

Everything below is ready to run. What is **not** done, because it needs your accounts, your money and a lawyer, is listed at the end.

What gets deployed

Three processes, deliberately separate:

Process	Job	Why separate
postgres	the database	managed service in production
web	serves the portal	must never do slow work
scheduler	crawls and maintains	must never serve a request

A long crawl cannot make the site slow, and restarting the site cannot interrupt a crawl.

Running it

```
# 1. Configuration. Never commit this file.
cp .env.example .env
#   set POSTGRES_PASSWORD, CONTACT_EMAIL, CONTACT_URL

# 2. Start
docker compose up -d --build

# 3. Create the schema
docker compose run --rm web alembic upgrade head

# 4. First crawl, to have something to show
docker compose run --rm web python -m auction_portal crawl baanknet --limit 200

# 5. Watch it
docker compose logs -f scheduler
curl localhost:8000/api/health
```

What runs by itself

Job	When	Purpose
Crawl each source	every 6 hours	new and changed listings
Refresh imminent auctions	hourly	auctions within 7 days move and get cancelled at short notice
Advance statuses	every 15 min	upcoming to live to closed, on time
Expire old listings	daily 03:30 IST	stop publishing concluded auctions
Health check	every 2 hours	shout when a source looks broken

Two scheduler settings matter for correctness. `coalesce` means a job missed during downtime runs once, not once for every interval that elapsed. `max_instances=1` means a slow crawl can never overlap the next run.

Monitoring

The failure that matters is **silent**: a changed CSS selector returns zero results without raising anything. So the alert is not "did it error" but "has it produced anything new lately".

```
docker compose run --rm web python -m auction_portal health
```

Exits non-zero when any source looks broken, so it works directly as a cron or uptime check. A source is unhealthy when it discovers nothing, or fetches pages but parses none of them, or more than half its fetches fail, or it has produced no new listing for 48 hours.

Wire the non-zero exit to whatever you use — a cron email, Healthchecks.io, Uptime Kuma, or Sentry picking up the `ERROR` log line.

Before going live

Required

1. **Legal review.** Non-negotiable. An Indian technology and IP lawyer should look at the scraping approach, the disclaimers, the privacy policy and the DPDP position. See Part E of the main document.
2. **A real `CONTACT_URL`.** It is sent with every request we make. It must resolve to a page saying who the bot is, what it collects, how often, and how to ask it to stop. A crawler that can be contacted usually gets left alone; an anonymous one gets blocked.
3. **A privacy policy and a takedown route**, with a named contact and a committed response time. Honour removal requests for concluded auctions without arguing.
4. **Managed PostgreSQL.** The compose file runs Postgres in a container, which is fine for staging. For production use RDS, Neon or Supabase in `ap-south-1`. Never self-host your source of truth.

5. **Backups**, tested by actually restoring one.

Recommended

- **HTTPS** via Caddy or nginx with Let's Encrypt.
- **Object storage** for `/app/data`. Cloudflare R2 has no egress fees.
- **Sentry** for errors.
- **Host in India** (`ap-south-1` or Azure India). Keeping Indian personal data in India removes a whole category of cross-border questions.

Cost

Item	Monthly
Small VPS or container host	Rs 800 – 2,000
Managed PostgreSQL	Rs 0 (free tier) – 4,000
Object storage	Rs 100 – 500
Domain	Rs 80
Total	Rs 1,000 – 6,500

Phase 1 needs no paid OCR, LLM or geocoding, because BAANKNET is a structured source. Those costs arrive with Phase 2 and the PDF work.

Scaling, when it is actually needed

Not yet. For reference:

Symptom	Change
Crawl too slow on one machine	Celery + Redis; the job functions do not change
Search slow past ~50k listings	Typesense alongside Postgres
Web slow under traffic	more <code>web</code> replicas behind a load balancer
Archive large	move <code>RawStore</code> to S3; the interface already fits

Do none of this before it hurts.